

UNIT 4 LEET CODE

1)Delete Duplicate Emails

Table: Person

Column Name	Type
id	int
email	varchar

Id is the primary key (column with unique values) for this table.

TASK:

Write a solution to delete all duplicate emails, keeping only one unique email with the smallest id.

SOLUTION:

```
1 # Write your MySQL query statement below
2 delete p1
3 from Person p1, Person p2
4 where p1.Email = p2.Email and p1.Id > p2.Id
```

Saved

☒ Testcase | [Test Result](#)

☒ Case 1

Input

Person =

id	email
1	john@example.com
2	bob@example.com
3	john@example.com

Output

id	email
1	john@example.com
2	bob@example.com

Activate Windows
Go to Settings to activate

Explanation:

john@example.com is repeated two times. We keep the row with the smallest Id = 1.

2)Customers Who Never Order

Suppose that a website contains two tables,the Customers table and the Orders table.

Table: Customers.

Id	Name
1	Joe
2	Henry
3	Sam
4	Max

Table: Orders.

Id	CustomerId
1	3
2	1

TASK:

Write a SQL query to find all customers who never order anything.

SOLUTION:

```
1 # Write your MySQL query statement below
2 select c.Name as Customers
3 from Customers c
4 where c.Id not in (select CustomerId from Orders);
```

Saved

☒ Testcase | [Test Result](#)

Input

Customers =

id	name
1	Joe
2	Henry
3	Sam
4	Max

Orders =

id	customerId
1	3
2	1

Output

Customers
Henry
Max

Explanation:

It finds the names of customers who have never placed an order. NOT IN checks that the customer's Id does not exist in the Orders.CustomerId list.

3)Biggest Single Number

Table my_numbers contains many numbers in column num including duplicated ones.

num
8
8
3
3
1
4
5
6

TASK:

write a SQL query to find the biggest number, which only appears once.

SOLUTION:

```
1 # Write your MySQL query statement below
2 select max(num) as num
3 from
4     (select num
5      from MyNumbers
6      group by num
7      having count(num) = 1) n
```

Saved

☒ Testcase | [Test Result](#)

Input

MyNumbers =

num
8
8
3
3
1
4
5
6

Output

num
6

Explanation:

The single numbers are 1, 4, 5, and 6. Since 6 is the largest single number, we return it.

4)Game Play Analysis I

Table: Activity

Column Name	Type
player_id	int
device_id	int
event_date	date
games_played	int

(player_id, event_date) is the primary key of this table.

This table shows the activity of players of some game.

Each row is a record of a player who logged in and played a number of games (possibly 0) before logging out on some day using some device.

TASK:

Write an SQL query that reports the first login date for each player .

```
1 # Write your MySQL query statement below
2 select a.player_id, min(a.event_date) as first_login
3 from Activity a
4 group by a.player_id
```

Saved

☒ Testcase | [Test Result](#)

Accepted Runtime: 122 ms

☒ Case 1

Input

Activity =

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

Output

player_id	first_login
1	2016-03-01
2	2017-06-25
3	2016-03-02

Explanation:

It finds each player's first login date from the Activity table. GROUP BY player_id groups each player, and MIN(event_date) gets their earliest login date.

5)Game Play Analysis II

Table: Activity

Column Name	Type
player_id	int
device_id	int
event_date	date
games_played	int

(player_id, event_date) is the primary key of this table.

This table shows the activity of players of some game.

Each row is a record of a player who logged in and played a number of games (possibly 0) before logging out on some day using some device.

TASK:

Write a SQL query that reports the device that is first logged in for each player.

```
select player_id, device_id
from Activity
where (player_id, event_date) in
      (select player_id, min(event_date)
       from Activity
       group by player_id);
```

Result table:

player_id	device_id
1	2
2	3
3	1

Explanation:

It finds the device used by each player on their first login date. The subquery gets each player's earliest event_date, and (player_id, event_date) IN matches that date to return the corresponding device_id.

6)Game Play Analysis III

Table: Activity

Column Name	Type
player_id	int
device_id	int
event_date	date
games_played	int

(player_id, event_date) is the primary key (column with unique values) of this table.

This table shows the activity of players of some games.

Each row is a record of a player who logged in and played a number of games (possibly 0) before logging out on someday using some device.

TASK:

Write a solution to report for each player and date, how many games played so far by the player. That is, the total number of games played by the player until that date. Check the example for clarity.

```
SELECT
    a.player_id,
    a.event_date,
    SUM(b.games_played) games_played_so_far
FROM
    Activity a LEFT JOIN Activity b ON
    b.player_id = a.player_id AND
    b.event_date <= a.event_date
GROUP BY
    a.player_id,
    a.event_date;
```

Output:

player_id	event_date	games_played_so_far
1	2016-03-01	5
1	2016-05-02	11
1	2017-06-25	12
3	2016-03-02	0
3	2018-07-03	5

Explanation:

For the player with id 1, $5 + 6 = 11$ games played by 2016-05-02, and $5 + 6 + 1 = 12$ games played by 2017-06-25.

For the player with id 3, $0 + 5 = 5$ games played by 2018-07-03.

Note that for each player we only care about the days when the player logged in

7)Exchange Seats

Table: Seat

Column Name	Type
id	int
student	varchar

- id is the primary key (unique value) column for this table.
- Each row of this table indicates the name and the ID of a student.
- The ID sequence always starts from 1 and increments continuously .

TASK:

Write a solution to swap the seat id of every two consecutive students. If the number of students is odd, the id of the last student is not swapped.

```
1 # Write your MySQL query statement below
2 SELECT
3     a.id id,
4     COALESCE(b.student, a.student) student FROM seat a LEFT JOIN seat b ON b.id = (CASE WHEN a.id%2=1 THEN a.id+1 ELSE a.id-1 END)
5 ORDER BY a.id
```

Saved

Ln 4, Col 1

☒ Testcase ☐ Test Result

☒ Case 1

Input

Seat =

id	student
1	Abbot
2	Doris
3	Emerson
4	Green
5	Jeames

Output

id	student
1	Doris
2	Abbot
3	Green
4	Emerson
5	Jeames

Activate Windows
Go to Settings to activate Windows.

Explanation:

Note that if the number of students is odd, there is no need to change the last one's seat.